Toniq 基本設計書 記入用仕様書

Nov 4, 2026 · @nay

本書の目的と使い方

本書は、基本設計書「楽曲生成システム Toniq」（KCSKA-R4SA-アプリオリ(01)、2026/10/19版）の空欄を埋めるための情報を、現在のソースコード（Electron + Vite + TypeScript + Tone.js）から整理したものです。

- 各節の見出しは基本設計書の章番号に対応しています。表の内容はそのまま転記できる形にしています。
- 「実装状況」は 2026/11/04 時点のコードに基づきます。未実装の機能は「予定」として書き分けています。
- 設計書側の誤記・章番号のずれ・チームで決める必要がある事項は、最終章にまとめています。

1〜2章 システム構築方針・制約条件

設計書の既存の記述は生かし、「どう実現しているか」を一文ずつ足すと完成します。

1.1 システム開発の狙い（追記案）

本システムは、ジャンルと雰囲気を選ぶだけで楽曲を自動生成できるルールベースの作曲アプリである。AI（機械学習）を使わず、音楽理論に基づくパターンDBと生成ルールで曲を組み立てる。

特徴	実現方法
生成後も編集が可能	シンプルモードの結果をカスタムモードに引き継ぎ（applySimpleToCustom）、パート差し替え・音量調整・パターン変更ができる
不協和音を少なくする	コード進行をディグリーで管理し、メロディはコード構成音とスケール音から生成。スケールロックでスケール外の選択を制限
著作権が生成者に帰属	既存曲の学習やサンプルを使わず、汎用的なコード進行・リズム型とシンセ音源（Tone.js）だけで生成する

2. 制約条件・前提条件（追記案）

- 対象OS：Windows（インストーラは NSIS 形式）。インターネット接続は不要（オフラインで動作）。
- 音声出力デバイス（スピーカー・ヘッドホン）があること。
- 拍子は 4/4、1小節 16 ステップ固定（3/4 は保留）。
- ジャンルは EDM／8bit／ローファイ／ゲームBGM／テクノの5種、雰囲気は 明るい／暗い／落ち着いた／切ない／激しいの5種。
- ターゲットは設計書どおり「音楽制作未経験者」「アイデアを得たい作曲経験者」。前者はシンプルモード、後者はカスタムモードが主な入口。

3章 システム概要

3.1 適用業務範囲

利用者が「曲の条件を選ぶ → 生成 → 試聴・編集 → ファイルに書き出す」までを範囲とする。

業務	対象画面	状況
雰囲気・ジャンル・テンポを選んで一曲生成	シンプルモード	実装済
BPM・キー・スケール・コード進行・ドラム・音量を細かく設定	カスタムモード	実装済
メロディの手入力	ピアノロール	予定
パートの配置（構成）	アレンジャー	予定
MIDI・WAV 書き出し	両モード	実装済（MP3 は予定）
作った曲の一覧・再生	ミュージックリスト	予定
音楽の知識クイズ	音楽クイズ	予定

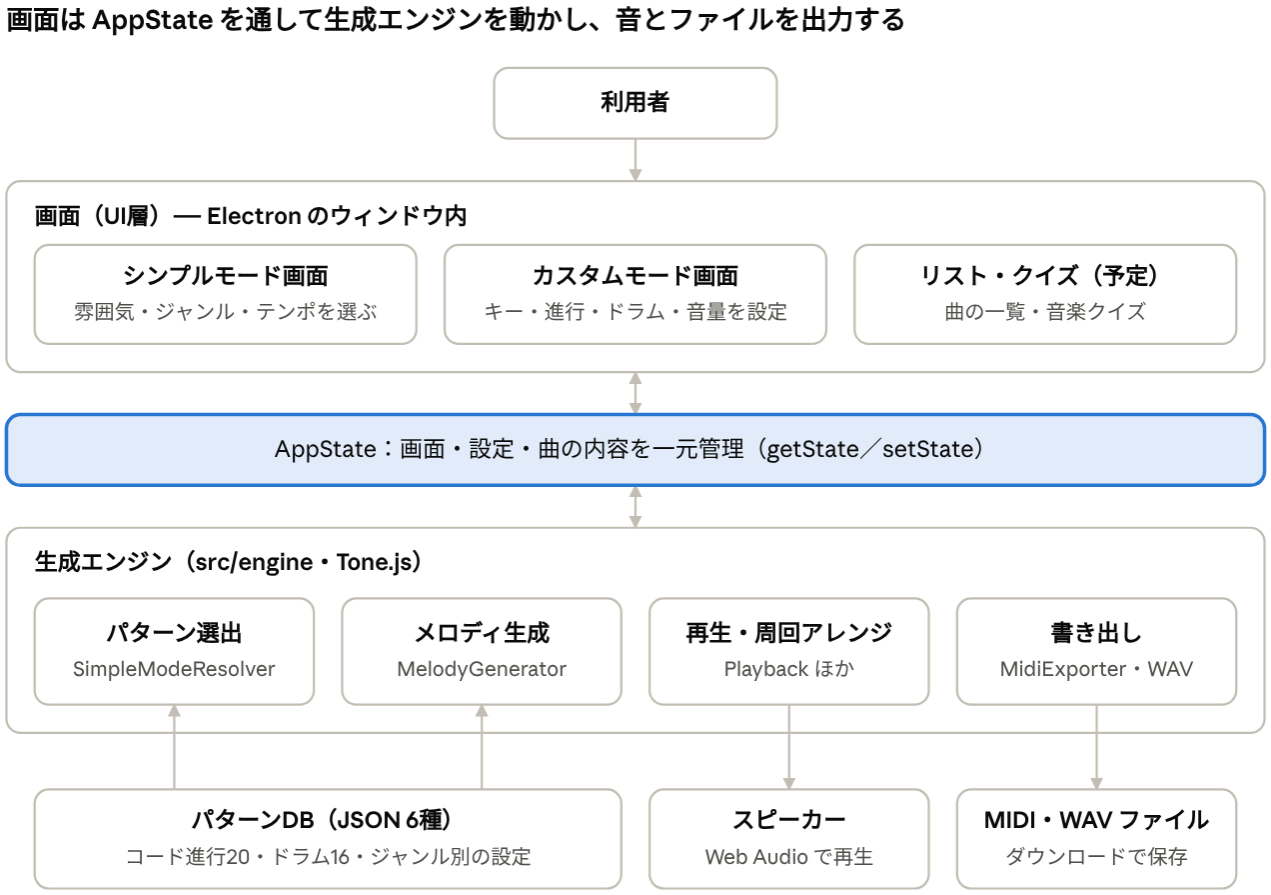
範囲外：歌詞・ボーカル、オーディオの録音、オンライン共有、ユーザー登録。

3.2 システム化の考え方

- **パターンDB＋ルールで生成**：コード進行 20 種・ドラム 16 種を JSON で持ち、ジャンル×雰囲気の表（easy_mode_table.json）から候補をランダムに選ぶ。ベース・アルペジオ・パッド・メロディはコード進行から自動で作る。

- **ディグリー管理**：コードは「Im・bVI・IV」のように度数で保存し、再生時にキーに合わせて実音へ変換する。キーを変えても同じ進行を使える。
- **周回アレンジ**：コード進行1周ごとにパートが増え、4周で全パートがそろう（ジャンル別の LOOP_SCHEDULE）。
- **2つの入口**：初心者はシンプルモードで三つ選んで生成、経験者はカスタムモードで細部を決める。シンプルモードの結果はそのままカスタムモードに引き継ぐ。
- **状態の一元管理**：画面・設定値はすべて AppState に置き、画面は AppState を描画するだけにする。

3.3 システム概要図



システム概要図·利用者・画面・状態・エンジン・データ・出力

画面は値を AppState に書き、エンジンは AppState とパターンDBを読んで音を鳴らす。この図の UI層・エンジン層・データの3段は、4.1 システム構成図の元にもなります。

4章 システム構成

[開発言語] TypeScript（画面は HTML / CSS）。サーバーを持たないスタンドアロンのデスクトップアプリ。

4.1 システム構成図

図に描く要素は次の3層。3.3 の図に「Windows PC」「Electron」の枠を重ねると構成図になります。

1. ハードウェア：Windows PC（スピーカー・ヘッドホン）
2. Electron：メインプロセス（ウィンドウ生成のみ、1200×800、最小 800×600）＋レンダラープロセス（Chromium）
3. アプリ：UI層（index.html・src/ui）→ エンジン層（src/engine、Tone.js）→ データ層（src/data の JSON）。音は Web Audio API 経由で出力、書き出しはブラウザのダウンロードとして保存

4.2 システム構成表

区分	構成要素	バージョン	用途
ハードウェア	Windows PC	Windows 10/11	実行環境
実行基盤	Electron	^44.2.0	デスクトップアプリ化
言語	TypeScript	^7.0.2	全ソースコード
ビルド	Vite＋vite-plugin-electron	^8.2.2 / ^1.1.2	開発サーバー・ビルド
配布	electron-builder	^26.15.3	Windows インストーラ（NSIS）作成
音声	Tone.js	^15.1.22	シンセ音源・シーケンス・エフェクト・WAV レンダリング
MIDI	@tonejs/midi	^2.0.28	MIDI ファイル作成
データ	JSON ファイル 6 種	—	パターンDB・設定値（12章）
開発ツール	VS Code、Git	—	開発・構成管理

5.1 機能一覧と実装状況

31機能のうち、実装済は 20、一部実装は 3、未実装は 8。「対応する実装」列は 5.2 の備考欄やクラス図を書くときに使えます。

No	ID	機能名	画面	状況	対応する実装
1	B01	BPM選択	カスタム	実装済	BPMスライダー 40〜220（CustomScreen）
2	B01	テンポ選択	シンプル	実装済	ゆっくり／普通／速い → tempo_presets.json（resolveTempoPreset）
3	B01	BPMクリック選択	両方	実装済	TapTempo（最大8回の間隔を平均、2秒空くとリセット、20〜240）
4	B01	BPM試聴	両方	未実装	（メトロノーム音が必要）
5	B01	小節数選択	カスタム	実装済	8/16/24/32/48/64 小節。Playback がその小節で自動停止
6	B01	ジャンル選択	両方	実装済	チップ（シンプル）／ドロップダウン（カスタム）5種
7	B01	ジャンル試聴	シンプル	未実装	—
8	B01	リード選択	カスタム	実装済	「インスト」：ソフトサイン／シンセリード／スクエア／プラック（MelodyPlayer）
9	K01	キー音選択	カスタム	実装済	ルート音 C〜B の12種。シンプルモードは A 固定
10	K01	スケール選択	カスタム	実装済	major/minor/phrygian/dorian（DBの base_scale）
11	K01	キー・スケール試聴	カスタム	未実装	—
12	C01	コード進行カテゴリ選択	カスタム	実装済	雰囲気5種でコード進行を絞り込み
13	C01	雰囲気選択	シンプル	実装済	チップ5種→ easy_mode_table.json からランダム選出
14	C01	コード進行選択	両方	実装済	カスタム：一覧から選択、シンプル：再抽選
15	C01	コード進行リズム選択	カスタム	実装済	全音符／2分／4分／8分（ChordPlayer）
16	C01	コード進行試聴	カスタム	実装済	進行1周分だけコードを鳴らす（Playback.preview）
17	D01	ドラムカテゴリ選択	カスタム	実装済	drum_patterns.json の genre タグ
18	D01	ドラムパターン選択	両方	実装済	カスタム：一覧、シンプル：再抽選
19	D01	ドラム試聴	カスタム	実装済	2小節分だけドラムを鳴らす

No	ID	機能名	画面	状況	対応する実装
20	S01	特殊音選択	カスタム	一部	パッド・アルペジオの ON/OFF のみ。FX（ライザーSE）は音量のみで ON/OFF ボタンなし
21	P01	メロディ入力	両方	未実装	「自分で入力」の選択肢とピアノロールタブの枠のみ
22	P01	スケールロック	両方	一部	現状は「コード進行を同じスケールに絞る」動作。ピアノロールへの制限は未実装
23	P01	ノーツ選択	両方	未実装	—
24	A01	パート配置	カスタム	一部	自動配置（LoopArranger）のみ。アレンジャー画面は未実装
25	A01	パート別音量調節	カスタム	実装済	7パートを 0〜100 で調整（VolumeManager、dB 変換）
26	A01	MIDIエクスポート	両方	実装済	Melody／Chord／Bass／Arpeggio の4トラック（MidiExporter）
27	A01	MP3エクスポート	両方	未実装	—
28	A01	WAVエクスポート	両方	実装済	Tone.Offline で曲全体をレンダリング（16bit PCM）
29	L01	楽曲再生	ミュージックリスト	未実装	生成した曲の再生・停止は両モード画面で実装済
30	Q01	音楽クイズ	音楽クイズ	未実装	—
31	G01	楽曲生成	両方	実装済	「生成する」ボタン（SimpleModeResolver・MelodyGenerator・LoopArranger）

機能IDは設計書の表でグループの先頭行にしか書かれていないため、同じグループの行に同じIDを入れています。5.2 で機能ごとに一意のIDが必要な場合は「B01-1〜B01-8」のように枝番を振る案を最終章に記載しています。

5.2 機能定義

各機能の定義表は、下の共通値を全機能に入れたうえで、機能ごとの表の内容を転記します。「状態」はアプリ内の AppState の項目名です。

全機能共通の記入値

項目	記入値
アクター	利用者
入力ー帳票、出力ー帳票	なし

項目	記入値
入力ーデータベース	なし（下表で JSON を読むものだけ記載）
出力ーデータベース	なし（保存機能がないため）
出力ーファイル	なし（書き出し機能だけ記載）

機能ごとの記入内容（No.1〜16）

No 機能名	事前条件	事後条件	入力	出力	機能説明・備考
1 BPM選択	カスタムモード画面が開いている	状態 bpm が更新され、tempoPreset が custom になる	画面：BPMスライダー（40〜220、1刻み）	画面：BPM値・情報バー。その他：再生中ならテンポを即時反映	スライダーでBPMを指定する。値は 20〜240 に丸める
2 テンポ選択	シンプルモード画面（生成前）	tempoPreset と bpm が更新される	画面：ゆっくり／普通／速いチップ。DB：tempo_presets.json	画面：選択中チップと BPM値	ジャンルごとの3段階からBPMを決める（例：EDM 115/128/140、ローファイ 70/80/95）。ジャンルを変えると同じ段階のBPMに追従
3 BPMクリック選択	シンプル／カスタムモード画面	bpm がタップ間隔から計算され、tempoPreset が custom	画面：「タップ」ボタンの連続クリック	画面：BPM値	2回目以降、直近最大8回の間隔の平均からBPMを算出。2秒以上空けるとリセット。20〜240 に制限

No 機能名	事前条件	事後条件	入力	出力	機能説明・備考
4 BPM試聴	BPMが決まっている	—	画面：BPM試聴ボタン	その他：メトロノーム音	未実装。現在のBPMでクリック音を数小節鳴らす想定
5 小節数選択	カスタムモード画面	bars が更新される	画面：小節数ドロップダウン（8/16/24/32/48/64）	画面：情報バーの小節数	曲全体の長さ。再生はこの小節数で自動停止。シンプルモードでは「進行の小節数×4周」を自動設定
6 ジャンル選択	シンプル／カスタムモード画面	genre が更新。プリセットなら bpm、カスタムでは drumCategory も追従	画面：ジャンルチップ／ドロップダウン（EDM・8bit・ローファイ・ゲームBGM・テクノ）	画面：選択状態・情報バー	ジャンルはテンポ・パターン候補・メロディの性格・ミックス・周回アレンジを決める
7 ジャンル試聴	シンプルモード画面	—	画面：試聴ボタン	その他：ジャンルの代表パターン音	未実装。ジャンルのドラム＋コードを2〜4小節鳴らす想定
8 リード選択	カスタムモード画面	melodyInstrument が更新	画面：「インスト」ドロップダウン	その他：メロディの音色	ソフトサイン（sine）／シンセリード（sawtooth）／スクエア／プラック。再生開始時に反映
9 キー音選択	カスタムモード画面	key が更新。自動生成メロディは作り直し	画面：ルート音ドロップダウン（C〜B）	画面：情報バーのキー、コード名	コード進行はディグリーで持つため、キーを変えるだけで移調できる

No 機能名	事前条件	事後条件	入力	出力	機能説明・備考
10 スケール選択	カスタムモード画面	そのスケールのコード進行に切り替わる	画面：スケールドロップダウン。DB：chord_progressions.json	画面：情報バー、コード進行一覧	選択肢は DB の base_scale（major/minor/phrygian/dorian）。同じ雰囲気の進行を優先
11 キー・スケール試聴	キーとスケールが決まっている	—	画面：試聴ボタン	その他：音階の音	未実装。スケール音を上行で鳴らす想定
12 コード進行カテゴリ選択	カスタムモード画面	mood が更新。選択中の進行が合わなければ候補の先頭に切替	画面：カテゴリドロップダウン（雰囲気5種）。DB：chord_progressions.json	画面：絞り込んだコード進行一覧	スケールロック ON のときは同じスケールの進行だけに絞る
13 雰囲気選択	シンプルモード画面（生成前）	mood が更新	画面：雰囲気チップ（明るい・暗い・切ない・激しい・落ち着いた）	画面：選択状態・情報バー	生成時、easy_mode_table.json の「ジャンル×雰囲気」の候補からコード進行とドラムをランダムに1つずつ選ぶ
14 コード進行選択	カスタム：画面を開いている。シンプル：生成済	chordPattern が更新。自動生成メロディは作り直し	画面：進行ドロップダウン／「再抽選」ボタン。DB：chord_progressions.json、easy_mode_table.json	画面：キーに合わせたコード名（例 Am-F-C-G）	再抽選は現在の進行以外から選ぶ。再生中なら先頭から再生し直す

No 機能名	事前条件	事後条件	入力	出力	機能説明・備考
15 コード進行リズム選択	カスタムモード画面	chordRhythm が更新	画面：リズムドロップダウン（全音符・2分・4分・8分）	その他：コードの発音間隔	全音符はコードの長さいっぱい伸ばす。それ以外は 8/4/2 ステップごとに刻む
16 コード進行試聴	コード進行が選択されている	試聴終了後に停止状態	画面：「▷ 試聴」ボタン	その他：コードの音（進行1周分）	曲を再生中なら止めてから鳴らす

機能ごとの記入内容（No.17〜31）

No 機能名	事前条件	事後条件	入力	出力	機能説明・備考
17 ドラムカテゴリ選択	カスタムモード画面	drumCategory が更新。合わなければ候補の先頭に切替	画面：カテゴリドロップダウン。DB：drum_patterns.json	画面：絞り込んだドラム一覧	アプリのジャンルを先頭に、DB にあるそれ以外のタグを後ろに並べる
18 ドラムパターン選択	カスタム：画面を開いている。シンプル：生成済	drumPattern が更新	画面：パターンドロップダウン／「再抽選」。DB：drum_patterns.json	画面：「#番号 パターン名」	16ステップのパターンをループ。再生中なら先頭から再生し直す
19 ドラム試聴	ドラムパターンが選択されている	試聴終了後に停止状態	画面：「▷ 試聴」ボタン	その他：ドラム音（2小節）	曲を再生中なら止めてから鳴らす
20 特殊音選択	カスタムモード画面	parts.pad／parts.arp（予定：parts.fx）が切り替わる	画面：ON/OFF ボタン	画面：ON/OFF 表示、音量行のグレー表示	パッド・アルペジオは実装済。ジャンル特有音（ライザー等の FX/SE）の ON/OFF は未実装。再生中に ON にすると、その周回で鳴るパートなら即座に鳴る

No 機能名	事前条件	事後条件	入力	出力	機能説明・備考
21 メロディ入力	メロディの入力方法が「自分で入力」	melodyNotes にノーツが追加される	画面：ピアノロール上のクリック	画面：ノーツ表示	未実装。ノーツの形式は自動生成と同じ（音名・長さ 1n〜16n・開始位置 小節:拍:16分）
22 スケールロック	—	scaleLock が切り替わる	画面：スケールロックスイッチ	画面：有効／無効、絞り込んだ一覧	現状：ON でコード進行を同じスケールに絞る。設計書の「音階を狭める」（ピアノロールの入力制限）は未実装
23 ノーツ選択	ピアノロールにノーツがある	選択ノーツの位置・長さが更新	画面：ノーツのクリック・ドラッグ、音符の長さ	画面：選択表示	未実装。機能一覧の説明（長さを選ぶ）と図（選んだノーツだけ動かす）が食い違うので要統一
24 パート配置	カスタムモード画面のアレンジャータブ	周回ごとのパート構成が更新	画面：アレンジャー上のパート選択・配置	画面：タイムライン	現状は LoopArranger がジャンル別の固定表で自動配置（1周目ドラム＋コード→4周目で全パート）。手動配置画面は未実装
25 パート別音量調節	カスタムモード画面	volumes が更新	画面：7パートのスライダー（0〜100）	画面：数値。その他：再生音量に即時反映	値は 20×log10(値/100) dB に変換。0 は無音。初期値：ドラム80・コード65・ベース70・アルペ60・パッド50・メロディ75・FX55
26 MIDI エクスポート	曲が生成されている	.mid ファイルが保存される	画面：MIDI ボタン	ファイル：toniq_export.mid	テンポ・4/4・Melody／Chord／Bass／Arpeggio の4トラック。ドラム・パッド・FX は含まない
27 MP3 エクスポート	曲が生成されている	.mp3 ファイルが保存される	画面：MP3 ボタン	ファイル：.mp3	未実装。WAV と同じレンダリング結果を MP3 エンコーダー（例：lamejs）で変換する想定
28 WAV エクスポート	曲が生成されている	.wav ファイルが保存される。再生中なら停止	画面：WAV ボタン	画面：「レンダリング中…」。ファイル：toniq_ジャンル_雰囲気（またはキー）.wav	曲の長さ＋2秒を Tone.Offline でレンダリングし 16bit PCM に変換。ON のパートだけ含める

No 機能名	事前条件	事後条件	入力	出力	機能説明・備考
29 楽曲再生	曲が生成（予定：保存）されている	再生中になる。小節数に達すると自動停止	画面：再生／停止ボタン（予定：ミュージックリストの曲）	その他：音声。画面：再生位置・周回数	生成画面での再生は実装済。ミュージックリストは曲の保存方法（12章）が決まってから
30 音楽クイズ	音楽クイズ画面	正誤が表示される	画面：クイズの種類、解答	画面：正誤判定	未実装。問題の種類・問題データは未定
31 楽曲生成	ジャンル・雰囲気・テンポが決まっている	chordPattern・drumPattern・melodyNotes・bars が決まり generated=true	画面：「✦ 生成する」ボタン。DB：easy_mode_table・chord_progressions・drum_patterns・melody_params	画面：生成結果（波形・各レイヤーの内容）。カスタムモードでは即再生	コード進行とドラムを候補から選び、メロディを生成。メロディは小節頭をコード構成音に着地させ、跳躍幅・休符率・音符の長さは melody_params.json に従う

6〜9章 ユースケース図・ロバストネス図・シーケンス図・クラス図

6章 ユースケース図

アクターは「利用者」1人。「6.1〜6.10 システム」の10枚は、機能IDのグループごとに1枚にするとただしく対応します（ロバストネス図の uc01〜uc07 とも一致）。

節	図名（案）	ユースケース
6.1	BPM・曲の基本設定（B01）	BPM選択、テンポ選択、BPMクリック選択、BPM試聴、小節数選択、ジャンル選択、ジャンル試聴、リード選択
6.2	キー・スケール（K01）	キー音選択、スケール選択、キー・スケール試聴
6.3	コード進行（C01）	カテゴリ選択、雰囲気選択、コード進行選択、リズム選択、試聴
6.4	ドラム（D01）	カテゴリ選択、パターン選択、試聴
6.5	特殊音（S01）	特殊音選択
6.6	ピアノロール（P01）	メロディ入力、スケールロック、ノーツ選択
6.7	アレンジ・書き出し（A01）	パート配置、パート別音量調節、MIDI／MP3／WAV エクスポート

節	図名（案）	ユースケース
6.8	楽曲再生（L01）	楽曲再生
6.9	音楽クイズ（Q01）	音楽クイズ
6.10	楽曲生成（G01）	楽曲生成（「コード進行選択」「ドラムパターン選択」を include）

7・8章 ロバストネス図・シーケンス図の不足分

機能	ロバストネス図	シーケンス図	描く内容（実装に合わせた流れ）
3 BPMクリック選択	なし	なし	利用者 → シンプル／カスタムモード画面 → タップボタンを押下（繰り返し）→ タップ間隔を平均 → BPMを決定・表示
29 楽曲再生	なし	あり	既存シーケンス図（楽曲再生画面）に合わせて「ミュージックリスト画面 → 楽曲を選択 → 再生」
30 音楽クイズ	なし	あり	音楽クイズ画面 → クイズの種類を選択 → 解答を選択 → 正誤判定
31 楽曲生成	なし	あり	生成ボタンを押下 → コード進行・ドラムを選出（パターンDBを参照）→ メロディを生成 → 生成結果を表示／再生

既存図と実装の差は次のとおりです。図を直すか、今後の実装で合わせるかを決めてください。

- BPM試聴・スケールロック・ノーツ選択・メロディ入力は図で「シンプルモード画面」からも使えるが、現在のシンプルモード画面には BPM試聴・スケールロックの部品がない。
- MIDI／MP3／WAV のロバストネス図はカスタムモード画面のみだが、シーケンス図と実装はシンプルモードからも書き出せる。
- シーケンス図 8.26〜8.28 の「生成後画面」は、実装では別画面ではなく同じ画面の中の表示切り替え（10章参照）。

9章 クラス図

クラスは TypeScript のモジュール（1ファイル＝1クラスとして扱う）を元に描きます。機能ごとの 9.1〜9.31 は、下の「使う機能No」で必要なクラスを拾ってください。

クラス（ファイル）	種類	主な属性	主な操作	使う機能No
SimpleScreen	境界	—	initSimpleScreen()、render()、generate()	2,3,6,13,14,18,26,28,29,31
CustomScreen	境界	—	initCustomScreen()、render()、setBpm()、chordCandidates()、drumCandidates()	1,3,5,6,8〜20,22,25,26,28,29,31
Router	境界	—	showScreen()、showTab()、render()	画面遷移全般
AppState	実体	screen, tab, mode, genre, mood, key, bpm, tempoPreset, bars, chordPattern, drumPattern, melodyNotes, melodyMethod, melodyInstrument, chordRhythm, scaleLock, drumCategory, parts, volumes	getState()、setState()、applySimpleToCustom()	全機能
ChordProgression／DrumPattern	実体	id, name, mood[], genre[], base_scale, length_bars, progression[]／steps, tracks	—	10,12,14,17,18
SimpleModeResolver	制御	—	resolveSimpleMode()、rerollChord()、rerollDrum()、resolveTempoPreset()	2,6,13,14,18,31
TapTempo	制御	tapTimes[]	tap()、resetTap()	3
MelodyGenerator	制御	—	generateMelody(chordPattern, key, genre)	31
DegreeResolver	制御	SCALE_INTERVALS, CHORD_INTERVALS	resolveChord(degree, type, inversion, key, scale, octave)	9,10,14,26
Playback	制御	—	play()、stop()、preview()、renderWav()、applyBpm()	1,3,16,19,28,29,31
LoopArranger	制御	LOOP_SCHEDULE, loopCount	startLoopArranger()、getActiveParts()、stopLoopArranger()	24,29
LayerManager	制御	—	setLayerEnabled()、reloadLayer()	20
VolumeManager	制御	Volumeノード	setVolume()、createVolumeNode()	25
MixerEngine	制御	リバーブ・コンプ・リミッター	initMixer()、applyMixPreset(genre)	6,29
Drum／Chord／Bass／Arp／Pad／Melody／Fx Player	制御	Synth、Part	init*Synth()*、*load*()、dispose*()	8,15,20,29
MidiExporter	制御	—	exportMidi(chordPattern, melodyNotes, key, bpm)	26

関連：境界クラスは AppState を読み書きし、制御クラスを呼ぶ。Playback → LoopArranger → 各 Player → VolumeManager／MixerEngine の順に音が流れる。

10章 画面設計

実装では「アレンジ画面」「ピアノロール画面」「生成後画面」は独立した画面ではなく、モード画面の中のタブや表示切り替えです。画面IDはそのまま使い、「実装上の位置」を備考に書くと整合します。

10.1 画面一覧（機能ID欄の記入案）

画面ID	画面名	機能ID	実装上の位置	状況
001	メニュー画面	—	起動時の画面（data-screen="menu"）。4枚のカード	実装済
002	シンプルモード画面	B01, C01, G01	simple 画面の設定タブ（生成前）	実装済
003	カスタムモード画面	B01, K01, C01, D01, S01, A01, G01	custom 画面の設定タブ	実装済
004	アレンジ画面	A01（パート配置）	custom 画面の「アレンジャー」タブ	準備中表示
005	ピアノロール画面	P01	simple・custom 画面の「ピアノロール」タブ	準備中表示
006	シンプルモード生成後画面	C01, D01, A01, L01	simple 画面の設定タブ（生成後、generated=true）	実装済
007	カスタムモード生成後画面	A01, L01	実装では切り替えなし（生成するとそのまま再生）	要検討
008	書き出し完了画面	A01	実装ではブラウザのダウンロードのみ。完了通知なし	未実装
009	ミュージックリスト画面	L01	data-screen="musiclist"	準備中表示
010	音楽クイズ画面	Q01	data-screen="quiz"	準備中表示

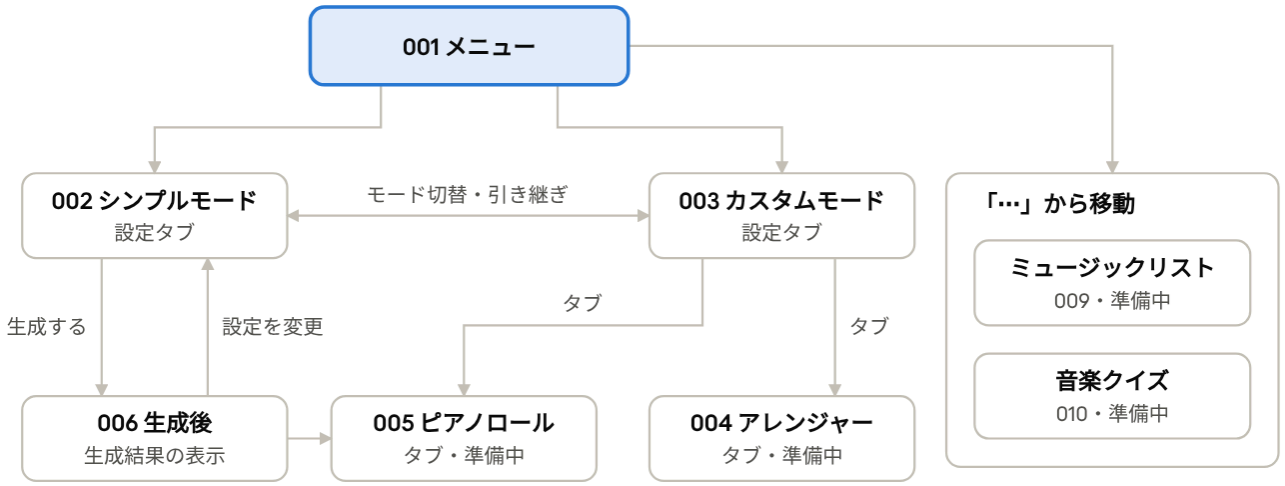
10.2 画面遷移図

下の図のとおり。遷移の条件は次のとおり。

- メニュー → 各画面：メニューのカード。各画面 → メニュー：ヘッダー「…」の「メニューに戻る」（リスト・クイズはパンくず「‹ メニュー」も）。

- シンプル ⇄ カスタム：ヘッダーの切替スイッチ、または「カスタムモードで編集する」。シンプル→カスタムでは設定を引き継ぐ。
- 002 → 006：「✦ 生成する」。006 → 002：「‹ 設定を変更」。
- 各モード画面 ⇄ 004・005：タブ（設定／アレンジャー／ピアノロール）。006 の「ピアノロールを開く」でも 005 へ。
- ミュージックリスト・音楽クイズはどの画面からも「…」メニューで移動できる。

メニューから4画面へ。生成後とタブは同じ画面の中で切り替わる



画面遷移図 · 画面ID 001〜010（007・008 は未定）

シンプルモードのピアノロールはタブからも開けます。007・008 は実装に画面がないため図に含めていません（最終章の決定事項4）。

10.3.2 画面項目定義（実装済画面）

番号は画面レイアウト（10.3.1）のスクリーンショットに振る番号として使えます。

画面	No	項目	種類	値・表示条件	操作時の挙動
共通	1	モード切替	ボタン2つ	シンプル／カスタム画面でのみ表示	そのモード画面へ
共通	2	「…」メニュー	ボタン＋メニュー	常に表示	メニュー／ミュージックリスト／音楽クイズへ
001	1〜4	モードカード	ボタン	シンプル・カスタム・リスト・クイズ	各画面へ
002	1	情報バー	表示	BPM（プリセット名）・ジャンル・雰囲気	—
002	2	雰囲気	チップ5	初期：明るい	mood を更新
002	3	ジャンル	チップ5	初期：EDM	genre と BPM を更新

画面	No	項目	種類	値・表示条件	操作時の挙動
002	4	テンポ	チップ3＋タップ	初期：普通。BPM値を併記	BPM を更新
002	5	メロディの入力方法	カード2	自動生成／自分で入力	melodyMethod を更新
002	6	生成する	ボタン	—	生成して 006 へ
002	7	カスタムモードで細かく設定	ボタン	—	003 へ（引き継ぎ）
006	1	再生／停止	ボタン	生成後に表示。再生中は「↺ 最初から」	先頭から再生／停止
006	2	生成された楽曲	波形風表示＋再生位置	拍ごとの棒。時間目盛り5点	—（実音声ではなく表示用）
006	3	メロディ差し替え	カード＋ボタン	「自動生成 フレーズA」等	再生成／ピアノロールを開く
006	4	コード差し替え	カード＋ボタン	コード名（例 Am - F - C - G）	別の進行を再抽選
006	5	ドラム差し替え	カード＋ボタン	「#番号 パターン名」	別のパターンを再抽選
006	6〜9	MIDI／WAV 書き出し、もう一度生成、カスタムで編集	ボタン	WAV は処理中「レンダリング中…」で無効化	各処理
003	1	情報バー	表示＋ボタン	BPM・キー・拍子・小節・周回（再生中）、停止・再生・生成する	—
003	2	BPM	スライダー＋タップ	40〜220	—
003	3	拍子	ドロップダウン	4/4（3/4 は準備中で選択不可）	—
003	4〜6	小節数・ジャンル・インスト	ドロップダウン	5.2 参照	再生中なら先頭から再生し直す
003	7	シンプルモードから引き継ぐ	ボタン	—	ジャンル・雰囲気からパターンとテンポを選び直す
003	8〜10	ルート音・スケール・スケールロック	ドロップダウン／スイッチ	ロック初期値：有効	—
003	11〜13	コード進行カテゴリ・進行＋試聴・リズム	ドロップダウン＋ボタン	進行は「コード名（進行名）」で表示	—
003	14・15	ドラムカテゴリ・パターン＋試聴	ドロップダウン＋ボタン	—	—
003	16・17	パッド／アルペジオ	ON/OFF ボタン	初期：ON	—

画面	No	項目	種類	値・表示条件	操作時の挙動
003	18	パート別音量	スライダー7本	メロディ・コード・ドラム・ベース・パッド・アルペジオ・FX/SE。OFF のパートはグレー	即時に音量反映
003	19	書き出し	ボタン2	MIDI／WAV	ファイル保存

11〜13章 帳票・データ・コード設計

11章 帳票設計

印刷する帳票はないため「該当なし」と書けます。出力物を帳票欄で扱う場合は次の表を使います。

帳票ID	帳票名	使用目的	機能ID	入出力区分
R01	MIDIファイル（.mid）	DAW での編集用：Melody／Chord／Bass／Arpeggio、テンポ、4/4	A01	出力
R02	WAVファイル（.wav）	聴取・配布用の音声（16bit PCMステレオ、曲の長さ＋2秒）	A01	出力
R03	MP3ファイル（.mp3）	同上（圧縮版）。予定	A01	出力

12章 データベース設計（JSON ファイル）

RDB は使わず、アプリに同梱した読み取り専用の JSON ファイルをテーブルとして扱います。利用者が作った曲は現在保存されません。

No	ID	テーブル名称	件数	説明・主な項目
1	T01	コード進行（chord_progressions.json）	20	id, name, mood[], genre[], bpm_range{min,max}, base_scale, length_bars（2/4/8）, progression[{bar, degree, type, inversion, duration_bars}], chord_rhythm, notes

No	ID	テーブル名称	件数	説明・主な項目
2	T02	ドラムパターン（drum_patterns.json）	16	id, name, genre[], mood[], bpm_range, steps（16）, step_unit（16n）, tracks{kick, snare, hihat_closed, hihat_open, clap, rim → pattern[16], velocity[16], note}
3	T03	シンプルモード表（easy_mode_table.json）	5×5	ジャンル→雰囲気→{bpm, chord_ids[], drum_ids[]}
4	T04	テンポプリセット（tempo_presets.json）	5	ジャンル→{ゆっくり, 普通, 速い} の BPM
5	T05	メロディパラメータ（melody_params.json）	5	ジャンル→{noteLengthWeights, restProbability, maxLeap, preferStepwise, octaveRange}
6	T06	ミックスプリセット（mix_presets.json）	5	ジャンル→{reverb, eq, compressor, limiter}
—	T07	楽曲（予定）	—	ミュージックリスト用。AppState の曲の内容（genre, mood, key, bpm, bars, chordPattern.id, drumPattern.id, melodyNotes, parts, volumes）＋曲名・作成日時。保存先は未定

ER図（12.2）に描く関係：T03 の chord_ids → T01.id、drum_ids → T02.id（多対多）。T03・T04・T05・T06 はジャンル名をキーに 1 対 1。予定の T07 は T01・T02 を 1 件ずつ参照。

13章 コード設計

13.1 業務コード

ID種別	説明	コード体系	備考
コード進行ID	T01 の主キー	chord_ + 3桁連番（chord_001〜）	
ドラムパターンID	T02 の主キー	drum_ + 3桁連番（drum_001〜）	画面では「#番号」で表示

ID種別	説明	コード体系	備考
ジャンル	曲のジャンル	EDM／8bit／ローファイ／ゲームBGM／テクノ	フォールバックはローファイ。DB にはシンセウェイブのタグもある
雰囲気	曲の雰囲気	明るい／暗い／落ち着いた／切ない／激しい	フォールバックは落ち着いた
テンポプリセット	テンポの段階	ゆっくり／普通／速い／custom	custom＝タップまたはスライダーで指定
キー	ルート音	C, C#, D, D#, E, F, F#, G, G#, A, A#, B	
スケール	音階	major／minor／phrygian／dorian	
ディグリー	コードの度数	ローマ数字＋m（短）、頭に b（半音下）。例 Im, bVI, IV, IIdim。ベース指定は _bassG 等	15種使用
コード種別	和音の種類	major, minor, dim, aug, maj7, min7, dom7	
パート	楽器パート	drum, chord, bass, arp, pad, melody, fx	表示名：ドラム〜FX/SE
コードリズム	コードの刻み	whole／half／quarter／eighth	
リード音色	メロディの音色	soft／lead／square／pluck	
音符長	ノーツの長さ	1n／2n／4n／8n／16n（Tone.js 表記）	

13.2 システム制御コード

ID種別	説明	コード体系	備考
画面	表示中の画面	menu／simple／custom／musiclist／quiz	AppState.screen
タブ	画面内のタブ	settings／arranger／pianoroll	AppState.tab
モード	利用中のモード	simple／custom	AppState.mode
メロディ入力方法	自動か手入力か	auto／manual	

ID種別	説明	コード体系	備考
生成済フラグ	生成前後の表示切替	true／false	AppState.generated
音量	パート音量	0〜100 の整数	dB に変換して使用

14〜17章 性能・運用・信頼性・保守設計

サーバーを持たない個人用アプリのため、運用・保守は「配布と改修」を中心に書くと実態に合います。数値目標は未計測のため、案として扱いチームで確定してください。

14章 性能設計

要求性能（案）	実現方法
「生成する」から結果表示まで1秒以内	DB は起動時にメモリへ読み込み済。選出とメロディ生成は数小節分の計算のみ
再生中に音が途切れない・リズムがずれない	Tone.js の Transport で音を先行スケジュール。画面更新は Tone.Draw で音と同期
音量・BPM 変更が即座に反映	Volume ノードと Transport.bpm を直接書き換える（再生し直し不要）
WAV 書き出しは曲の実時間より短い時間で完了	Tone.Offline で実時間を待たずにレンダリング
動作環境：一般的なノートPC	外部音源ファイルを持たずシンセで発音。ウィンドウ最小 800×600

15章 運用設計

- **15.1 業務内容**：インストーラ（npm run electron:build で release/ に作成）を配布 → 利用者がインストールして起動。パターンの追加は JSON を編集して再ビルド。
- **15.2 管理項目**：アプリのバージョン（package.json の version、現在 1.0.0）、パターンDBの件数と ID、ジャンル追加時にそろえるファイル（4つの JSON と LoopArranger の LOOP_SCHEDULE）。
- **15.3 運用体制**（任意）：リーダ：根井、メンバー：西薗・小手川。担当の割り振りはチームで記入。
- **15.4 運用スケジュール**：卒業研究の提出・発表日程に合わせて記入（本書では不明）。

16章 信頼性設計

節	記入案
16.1 データバックアップ	利用者データは保存しないため対象なし。書き出した MIDI／WAV は利用者が管理。パターンDB（JSON）は Git で履歴管理
16.2 データリストア	JSON を Git の履歴から戻して再ビルド
16.3 システムバックアップ	ソースコード一式を Git リポジトリで管理。配布したインストーラをバージョンごとに保管
16.4 システムリストア	アプリを再インストール。不具合時は前バージョンのインストーラに戻す

ミュージックリストで曲を保存するようになったら、16.1・16.2 にその保存先のバックアップ方法を追記します。

17章 保守設計（任意）

- **17.1 監視**：常時監視はなし。動作ログは開発者ツールのコンソールに日本語で出力（周回・パート ON/OFF・音量変更・書き出しエラー等）。
- **17.2 障害対応**：利用者が不具合を報告 → 担当者が再現・修正 → リーダーが確認して新バージョンを配布。現状、WAV 書き出しの失敗はコンソールに出るだけで画面には通知されない（改善候補）。未定義のディグリーが DB に入ると実行時に例外になるため、パターン追加時は degree_resolver の定義を確認する。

文書の不備・チームで決めるべき事項

そのまま直せる不備

- [ ] ヘッダーが「KCSKA-R4-チーム名(01)」のまま。表紙の文書番号「KCSKA-R4SA－アプリオリ(01)」と奥付の「KCSKA-R4-Toniq(06)」も食い違うので統一する。
- [ ] 目次と本文の章番号がずれている（目次：6 画面設計、7 帳票…、本文：6 ユースケース図、7 ロバストネス図、8 シーケンス図、9 クラス図、10 画面設計…）。目次を更新する。
- [ ] 目次に「5.2.1 ＪＳＵ見積機能」「保守契約機能」「障害対応機能」と「エラー! ブックマークが定義されていません」が残っている。本文の 5.2.32 保守契約機能・5.2.33 障害対応機能もテンプレートの名残りなので削除。
- [ ] 機能一覧 No.5 の説明「楽曲の小説数」→「小節数」。
- [ ] ロバストネス図／シーケンス図の「トップダウンリスト」→「ドロップダウンリスト」（7.6・8.6）。

- [ ] 7.2 の図が 7.3 の見出しの上（ページ境）にあり、7.3 クリック選択の図が無いように見える。図名も「BPMクリック選択機能」に統一。
- [ ] 図の置き場「.midに適用する」→「.midで出力する」など、文言の揺れをそろえる。
- [ ] 10.3.2 の注記が「6.3.1 画面レイアウト」を参照している →「10.3.1」。
- [ ] 改訂履歴・承認欄の責任者名が空欄。

チームで決めること

1. **未実装機能の扱い**：BPM試聴・ジャンル試聴・キースケール試聴・メロディ入力・ノーツ選択・MP3・ミュージックリスト・クイズを設計書に「予定」として残すか、スコープから外すか。
2. **機能IDの付け方**：グループID（B01 等）のままにするか、枝番（B01-1、B01-2 …）で機能ごとに一意にするか。
3. **スケールロックの意味**：現状の「コード進行を同じスケールに絞る」と、設計書の「ピアノロールでスケール外の音を入力させない」のどちらか、または両方か。
4. **生成後画面・書き出し完了画面**：画面一覧の 006〜008 を独立画面として作るか、実装に合わせて「画面内の状態」と書き換えるか。
5. **曲の保存先**：ミュージックリスト用に、アプリ内保存（localStorage など）かファイル保存か。12・16章の記述がこれで決まる。
6. **小節数の仕様**：現状はアレンジが最大8周で、曲の長さは「小節数」で止めるだけ。「小節数＝曲全体の長さ」の設計どおりにどこまで実装するか（書き出しの長さも含む）。
7. **特殊音の内容**：「ジャンル特有の音」を現状の FX（ライザー）とパッド・アルペジオの ON/OFF で良いとするか、ジャンルごとの音（8bit のコイン音など）を追加するか。